

Tilsynsapp-PNB

Web application for conducting and archiving building site visits (befaringer) in Norwegian municipalities. Inspectors fill in structured checklists, generate site visit reports, and send them directly to the municipality's records management system (Plan & Building / Public 360°) via the SIF API.

Contents

- [Features](#)
 - [Technology](#)
 - [Getting started](#)
 - [Environment variables](#)
 - [Database migrations](#)
 - [Architecture](#)
 - [SIF integration](#)
 - [Archival flow](#)
 - [Admin functionality](#)
 - [Accessibility](#)
 - [Security](#)
-

Features

Inspectors

Feature	Description
New site visit	Multi-step form with case reference, property data, and measure type
Survey area & measure type	Multi-select classification pulled directly from PNB code tables «eBy Supervision area» and «eBy Measure type» — used to filter which checkpoints are shown
Structured checklist	100+ checkpoints organised by category, dynamically filtered by measure type, property attributes, survey area, and measure type from PNB
Finding registration	Record status per checkpoint (ok / deviation / not checked) with free-text comment, responsible contact, and GPS coordinates
Attachments	Upload photos and documents to individual checkpoints or to the site visit as a whole. Photos containing GPS EXIF data (e.g. taken with a mobile camera with location services enabled) have their address automatically stamped onto the image (reverse geocoding via Nominatim); falls back to raw coordinates if the lookup fails
PDF report	Generate a professional site visit report with case metadata, checklist findings, deviation summary, inline images, and map
Archiving	Send a completed site visit to the municipal records system (Public 360°) — create a new document or update an existing one

Dashboard	Overview of active and archived site visits with filtering by status, date, and property
My PNB cases	Dedicated dashboard tab that fetches all PNB cases where the logged-in user is the responsible case officer — shows properties, contacts, case stages, and deadlines
Case file viewer	Files tab on each site visit shows all documents from the linked PNB case. PDF files open in an inline iframe viewer (bytes fetched via proxy → Blob → <code>URL.createObjectURL()</code>); images open in a lightbox; other file types download directly
Map picker	Select coordinates for the site visit location via an interactive OpenStreetMap map
Dark/light theme	Automatic system theme with manual override (light / dark / system)

SIF integration (Plan & Building / Public 360°)

- Case search and lookup of property and parties directly from PNB
- Auto-load case stages (behandlingstrinn) based on case number; inspector selects a stage in the archive panel to link the document to that stage in 360° via `AdditionalFields.ToStage`
- Synchronisation of external participants (e.g. fire safety officer) into 360° as contact persons
- Automatic document creation with uploaded PDF report, JSON metadata export, and attachments
- **Update existing document** — fetch documents already on the case and add a new version
- Automatic dispatch (auto-dispatch) of documents to recipients after archiving
- Supports two authentication modes:
 - **AuthKey** — simple key-based access (GUID header)
 - **OAuth2 Client Credentials (combined_daemon)** — Azure AD enterprise authentication

Technology

Layer	Choice
Framework	Next.js 15 (App Router, TypeScript strict mode)
UI library	React 19
Styling	Tailwind CSS v4 with dark mode
Internationalisation	next-intl (Norwegian / English via cookie)
Database	Supabase (PostgreSQL)
Authentication	Supabase Auth (email + password, JWT, cookie sessions via <code>@supabase/ssr</code>)
File storage	Supabase Storage
PDF generation	jsPDF + jspdf-autotable + pdf-lib (attachment merging)

Map tiles	OpenStreetMap (client-side Leaflet) + Kartverket WMS (server-side static map)
External integration	SIF API – Public 360° (RPC over HTTPS)
Linting / formatting	Biome (replaces ESLint + Prettier)
Package manager	Bun
Deployment	Vercel
Testing	Jest
Code quality	SonarQube (sonar-project.properties)

Getting started

Prerequisites

- [Bun](#) (replaces Node.js / npm)
- A Supabase project (or local Supabase via `supabase start`)

Installation

```
git clone <repo-url>
cd Tilsynsapp-PNB
bun install
cp .env.example .env.local # Fill in values – see table below
bun run dev                # Starts at http://localhost:3000
```

Available commands

```
bun run dev          # Development server
bun run build        # Production build
bun run start        # Start production server
bun run lint         # Biome (check + fix)
bun run format       # Biome (format only)
bun run typecheck    # TypeScript type check (tsc --noEmit)
bun run test         # Run tests
bun run test:watch   # Tests in watch mode
bun run test:coverage # Test coverage report
bun run db:generate  # Generate Supabase TypeScript types from schema
bun run set-admin    # Grant admin access to a user (by email)
```

Environment variables

Copy `.env.example` to `.env.local` and fill in the values.

Supabase

Variable	Description
NEXT_PUBLIC_SUPABASE_URL	URL to the Supabase project
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public anon key (safe to expose to browser)
SUPABASE_SERVICE_ROLE_KEY	Service role key — server only, never expose to browser

SIF / Public 360°

Variable	Description
SIF_BASE_URL	Base URL of the 360° instance, e.g. <code>https://customer.public360online.com</code>
SIF_RPC_PATH	RPC path, typically <code>/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC</code>
SIF_AUTH_MODE	<code>authkey</code> or <code>combined_daemon</code>
SIF_AUTHKEY	API key (GUID) for <code>authkey</code> mode
SIF_BEARER_TOKEN_URL	Azure AD token endpoint (<code>combined_daemon</code>)
SIF_CLIENT_ID	OAuth2 client ID sent as <code>clientId</code> header
SIF_CLIENT_SECRET	OAuth2 client secret
SIF_CLIENT_ID_OAUTH	OAuth2 application ID used for token requests
SIF_SCOPE	OAuth2 scope, e.g. <code>api://.../.default</code>
SIF_TIMEOUT_MS	API call timeout in milliseconds (default: <code>30000</code>)

Other

Variable	Description
NEXT_PUBLIC_APP_URL	Production URL (used by Vercel)

SIF settings can also be configured via the admin interface (`/dashboard/admin/sif-config`). Settings stored in the database take precedence over environment variables. An in-process cache (60 s TTL) prevents repeated DB round-trips on every RPC call.

Database migrations

Migrations are in `supabase/migrations/` and run via the Supabase CLI:

```
supabase db push      # Apply migrations to remote project
supabase db reset     # Reset local database and re-apply all migrations
```

Migration history

File	Contents
------	----------

001_initial.sql	Core tables: inspections, inspection_answers, attachments, list_items
002_sif_settings.sql	SIF configuration table (sif_settings)
005_external_participants.sql	External participants (people not in the PNB case)
010_sif_stage.sql	Case stage field on inspections
013_auto_dispatch.sql	Auto-dispatch flag in SIF settings
015_checkpoint_definitions.sql	Dynamic checkpoint library in the database
017_applicant_recno.sql	Applicant 360° recno stored on site visit
020_audit_log.sql	Audit log for admin actions (ISO 27001 A.12.4.1)
021_befaring_rename.sql	New fields befaringsomrade text[] and tiltakstype text[] on inspections; applies_to_omrade and applies_to_type_codes ON checkpoint_definitions
022_measure_type_optional.sql	measure_type_id made nullable after removal of Step 2 in the new inspection flow
023_checkpoint_omrade_mapping.sql	Sets applies_to_omrade on checkpoints based on subject area (first draft, superseded by 024)
024_checkpoint_full_mapping.sql	Full mapping against confirmed SIF code table: fixes incorrect names from 023, sets all supervision areas and applies_to_type_codes for facade, change-of-use, demolition, block-building, and VA-specific checkpoints
025_checkpoint_tiltakstype_extended.sql	Extended measure-type mapping based on PBL, TEK17 and SAK10 — rolled back due to null constraint error
026_fix_missing_checkpoints_and_025.sql	Inserts 5 missing checkpoints (UK001, PRD001, KS001, LB001, NB001) that only existed in TypeScript and re-applies all 025 updates using explicit ARRAY values
027_inspection_avvik Frist.sql	Adds avvik Frist DATE to inspections — deadline for closing deviations, per SAK10 § 15-3
028_answer Frist.sql	Adds Frist DATE to inspection_answers — per-deviation correction deadline, shown in checklist, HTML report and PDF

Architecture

```

app/
├─ api/                                # Next.js Route Handlers (server-side)
│   └─ archive/                        # POST /api/archive – full archival orchestration
│       └─ preview/                   # POST /api/archive/preview – preview without
archiving

```

```

|   ├── inspections/           # CRUD for site visits + answers + attachments
|   ├── inspection-codetables/ # GET – fetch survey area / measure type from PNB code
table
|   ├── admin/                 # Admin endpoints (require is_admin = true)
|   |   ├── users/             # User management
|   |   ├── checkpoints/       # Checkpoint definitions
|   |   ├── inspection-config/ # Configurable lists (background reasons)
|   |   ├── archivals/         # Archival log
|   |   └── sif/                # SIF proxy endpoints (20+)
|   |       ├── case-lookup/    # Single case lookup (caseNumber / uid / externalId)
|   |       ├── case-search/    # Search cases by partial number or title
|   |       ├── case-contacts/  # Parties on a case
|   |       ├── case-stages/    # Case stages (behandlingstrinn)
|   |       ├── case-documents/ # Documents already on a case
|   |       ├── case-estates/   # Properties linked to a case
|   |       ├── enterprise-search/ # Company search (GetEnterprises)
|   |       └── my-cases/       # GET – fetch PNB cases where user is responsible
(filtered by ResponsiblePersonName)
|   |   ├── pnb-case/[recno]/   # GET – fetch a single PNB case with full detail
(contact, stages, milestones)
|   |   ├── code-tables/        # Archive codes, categories, statuses (admin only)
|   |   ├── settings/           # Read current SIF config
|   |   ├── health/             # Connectivity check
|   |   └── debug-raw/          # Raw RPC call for testing
└── dashboard/                  # Authenticated pages
    ├── inspections/[id]/       # Site visit workspace (checklist, attachments,
archive)
    ├── inspections/new/        # New site visit form
    ├── pnb-cases/[recno]/      # Detail page for a single PNB case (contacts, stages,
milestones)
    |   ├── admin/              # Administration pages
    |   |   ├── checkpoints/    # Checkpoint editor (inc. codetable filter config)
    |   |   ├── users/          # User management
    |   |   ├── sif-config/     # SIF settings form
    |   |   ├── sif-test/       # SIF connection tester
    |   |   ├── tilsyn-config/  # Site visit config (background-reason list)
    |   |   └── archivals/      # Archival audit log
    └── login/                  # Login page

lib/
└── sif/                        # SIF API client and services
    ├── client.ts               # Low-level RPC dispatcher (auth, retry, error
mapping)
    ├── auth.ts                 # AuthKey / OAuth2 token management (with caching)
    ├── settings.ts             # Load + cache SIF config from DB or env vars
    ├── archival.ts             # Archival orchestrator (case → upload → document)
    ├── case-service.ts         # GetCases (lookup + search)
    ├── contact-service.ts      # SynchronizeContactPerson, GetEnterprises
    └── document-service.ts     # CreateDocument, UpdateDocument, GetDocuments,
DispatchDocuments
    ├── file-service.ts         # FileService/Upload (single + batch)
    └── errors.ts               # Typed domain errors (SifCaseNotFoundError, etc.)

```

```

|   ├── types.ts                # Full SIF API type definitions (~1000 lines)
|   └── extensions/             # Higher-level utilities (not integrated into app)
|       ├── referred-cases.ts   # Fetch referring cases and their approved documents
|       ├── file-download.ts    # Download files from 360° via SIF auth
|       ├── user-service.ts     # UserService/GetUsers
|       └── search-service.ts   # SearchService/Search + document/case search
└── pdf/
    ├── generate.ts             # PDF report generator (jsPDF) + JSON export
    ├── map-image.ts            # Fetch static map from Kartverket WMS
    └── map-capture.ts          # Client-side OSM tile capture (canvas → JPEG)
└── checklist/
    └── filter-engine.ts        # Checkpoint filtering, grouping, summary calculation
└── i18n/                       # Norwegian/English translations (next-intl
compatibility adapter)
└── supabase/                  # Supabase server/browser client factories
└── api-auth.ts                # JWT guards: requireUser() / requireAdmin()
└── audit-log.ts               # Structured admin action logging
└── legal-reference.ts         # Lovdata URL builder from legal reference strings

config/
└── sif-mapping.ts             # Document archive codes, contact roles, title
template

data/seed/
└── checkpoint-definitions.ts   # 100+ checkpoint definitions with legal references
└── measure-types.ts           # 10 building measure types (enebolig, tilbygg, etc.)

types/
└── index.ts                   # Core domain types (Inspection, CheckpointDefinition,
etc.)

i18n/
└── request.ts                 # next-intl server config (locale from cookie)

messages/
└── nb.json                    # Norwegian translations (ICU format)
└── en.json                    # English translations (ICU format)

biome.json                     # Biome linting + formatting config (replaces ESLint +
Prettier)
sonar-project.properties       # SonarQube project configuration

```

SIF integration

Authentication

Two modes are supported, configured via `SIF_AUTH_MODE` :

authkey — the API key (GUID) is appended as a query parameter:

POST

`https://customer.public360online.com/Biz/v2/api/call/SI.Data.RPC/CaseService/GetCases?`

```
authkey=<guid>
```

combined_daemon — OAuth2 Client Credentials Grant (Azure AD):

- Fetches a bearer token from the configured token endpoint
- Token is cached in-process with a 60-second buffer before expiry
- Sends `Authorization: Bearer <token>` + optional `ClientID` header

Rate limiting and retry

The RPC client (`lib/sif/client.ts`) automatically retries on HTTP 429 with exponential backoff:

- Retry 1: 2 s
- Retry 2: 4 s
- Retry 3: 8 s
- Respects `Retry-After` header if present

SIF mapping configuration

`config/sif-mapping.ts` contains the document archive codes that must be configured for each 360° installation. Replace the placeholder values before deployment:

```
inspectionReport: {
  archive: "recno:2",          // TODO: Replace with correct archive recno
  category: "recno:111",      // TODO: Replace with correct category
  status: "J",                // J = Journalført
  titleTemplate: "{{title}} - Tilsynsrapport - {{date}}",
  mainFileRelationType: "H",  // H = main document
  attachmentRelationType: "V", // V = attachment
},
contactRoles: {
  applicantRecipientRole: "Mottaker",
  copyRecipientRole: "Kopi til",
},
```

Find the correct values using `SupportService/GetCodeTableRows` or the 360° admin portal. All values can also be overridden at runtime via the admin UI (`/dashboard/admin/sif-config`).

Title template variables

The document title in 360° is generated from a configurable template. Available variables:

Variable	Description
{{propertyAddress}}	Property address from the inspection
{{caseNumber}}	Case number from PNB
{{title}}	Case title from PNB
{{date}}	Inspection date (DD.MM.YYYY)
{{year}}	Inspection year (YYYY)
{{inspectorName}}	Name of the inspector

{{applicantName}}	Applicant (søker) name
{{gnrBnr}}	Land register number (gnr/bnr)
{{measureType}}	Measure type (tiltakstype)
{{inspectionId}}	Internal inspection UUID

Example: {{title}} - Tilsynsrapport - {{date}}

Archival flow

The archival endpoint (`POST /api/archive`) orchestrates the following steps, optimised with parallel execution:

- (parallel)
 - Start SIF case lookup (`findCaseInSif`) – needs only request body data
 - Load inspection + answers + attachments from DB
 - (after DB load, parallel)
 - Download attachment files from Supabase Storage
 - Fetch static map image from Kartverket WMS
- (parallel)
 - Generate PDF report (`jsPDF` + inline images)
 - Insert pending archival record in DB
- `archiveInspectionToSif()` – (parallel)
 - Upload files to SIF (PDF + JSON + attachments)
 - Sync external participants via `SynchronizeContactPerson`
- Create (or update) document in 360° via `DocumentService/CreateDocument`
- (optional) Auto-dispatch document to recipients
- (parallel)
 - Update archival record in DB (success / failed)
 - Update site visit status to "archived"

The pre-fetched SIF case (step 1) is passed directly to `archiveInspectionToSif()` to skip a redundant lookup.

Admin functionality

Admin users have access to `/dashboard/admin/` :

Page	Function
Users	Create users, grant/revoke admin access, update display names
Checkpoints	View, edit, activate/deactivate checkpoint definitions; configure <code>applies_to_omrade</code> and <code>applies_to_type_codes</code> for code-table-based filtering

Site visit configuration	Configure the background-reason list (survey area and measure type are fetched from PNB code tables)
SIF configuration	Configure SIF endpoint, authentication, archive mapping, and title template
SIF test	Test SIF connectivity and inspect raw RPC responses
Archival log	All archival attempts with status, document numbers, and error messages

Audit logging

All admin actions are automatically logged to the `audit_logs` table (ISO 27001 A.12.4.1):

Action	Trigger
<code>user.create</code>	New user created
<code>user.set_admin</code>	Admin access granted or revoked
<code>user.update_name</code>	Display name updated
<code>checkpoint.create/update/deactivate/delete</code>	Checkpoint definition changed
<code>inspection_config.create/delete</code>	Dropdown list item added or removed
<code>sif_settings.update</code>	SIF settings saved (sensitive fields excluded from log)

Accessibility (WCAG 2.1 AA)

- **Skip-to-content link** — keyboard focus jumps directly to main content (WCAG 2.4.1)
- **Colour contrast** — all text meets the 4.5:1 ratio in both light and dark themes
- **Visible focus indicators** — `focus-visible` rings on all interactive elements
- **ARIA attributes** — `aria-expanded` , `aria-haspopup` , `aria-pressed` , `aria-live` , `role="dialog"` , `role="alert"` used throughout
- **Semantic HTML** — `<nav aria-label>` , `role="menu"` , `role="listbox"` , `role="group"` on relevant elements
- **Keyboard navigation** — dropdowns and modals close with the Escape key
- **Screen reader support** — `aria-label` on all icon-only buttons, `aria-hidden` on decorative elements

Security

- **Row Level Security (RLS)** on all Supabase tables — users can only access their own data
- **JWT validation** on every API route via `requireUser()` / `requireAdmin()`
- **Admin role** stored in `app_metadata.is_admin` — can only be set server-side via the service role key
- **Secrets never logged** — `authkey`, `client_secret`, and bearer tokens are masked in all log output
- **CSP headers** configured in `next.config.mjs`
- **HSTS**, `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff`
- **SIF credentials** stored encrypted in Supabase DB — never exposed to the browser
- **Audit log** captures all privileged actions with user ID and timestamp